Kaggle Credit Card Fraud Detection 기반

신용카드 사기 탐지(FDS) 프로젝트 보고서

작성자: 홍승현

날짜: 2026-04-05

SSOT: fds_pipeline.py · 시연: app.py

목 차

Table of Contents

제1장 데이터 개요
변수 구조 / 클래스 / 기초 통계

제2장 탐색적 데이터 분석(EDA)
불균형 / 상관 / Time / Amount

제3장 전처리 파이프라인 (SSOT)
sin/cos · RobustScaler · 단계별 입출력

제4장 데이터 분할과 리샘플링
층화 분할 / SMOTETomek / 누출 논의

제5장 모델링과 실험
베이스라인 / 1차 벤치마크 / 2차 실험 / 전처리 민감도

제6장 평가 지표와 임계값
Recall / F2 / PR-AUC / threshold 역할

제7장 Streamlit 앱과 SHAP 설명
아티팩트 / 시연 화면 / Explainer 구현

제8장 한계와 향후 계획
OOT / 익명화 / 프로토타입 한계

부 록 실행 명령어 · 파일 맵 · 보고 그림 대응 · 참고 링크

프로젝트 요약

Executive Summary

목적: **Kaggle Credit Card Fraud Detection** 데이터로 이진 분류 FDS 파이프라인 프로토타입 구축

SSOT: **fds_pipeline.py** - 모든 실험이 동일 전처리 함수를 참조 (재현성 보장)

- 전처리: Time→sin/cos(86,400s), Amount→RobustScaler, SMOTETomek은 Train에만
- 실험: 베이스라인 XGB → 1차 벤치마크(최대 5종) → 2차 실험(val 기반 임계값 선택)

지표 중심: **Recall / F2 / PR-AUC** (Accuracy 단독 지양)

- 시연: Streamlit app.py + SHAP Waterfall (predict_proba 래핑 Explainer)

한계·보완: 스케일 누출 논의·OOT 한계는 4-3·8-1에, 완화·향후는 5-8·8-2에 명시

284,807건	0.17%	31개	8:2
총 거래	사기 비율	변수	Train:Test

제1장

데이터 개요

출처 · 변수 구조 · 클래스 · 기초 통계

1-1 데이터 출처와 구성

Dataset Overview

1	Time	Amount	Class	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	V11	V12	V13	V14
2	10765	8	0	-0.30632	0.794015	1.767337	-0.67501	0.039556	-0.31608	0.33689	-0.05978	0.68996	-0.89917	2.136549	-1.47209	2.706664	1.432008
3	18489	21.99	0	-1.54441	0.192998	0.67199	-0.62573	1.953606	0.231276	-0.01792	0.125894	1.19821	-0.72231	0.789475	-3.035	0.568386	1.287709
4	19914	5	0	1.405693	-0.45506	0.640889	-0.24835	-1.06802	-0.78404	-0.68229	-0.22092	0.912998	0.136482	0.275076	-3.08936	1.213983	1.05926
5	25208	7.3	0	-1.92445	1.582919	-0.31336	-2.90643	2.267501	3.153706	0.398868	0.587037	1.481131	1.907915	0.074545	-0.35702	-0.14162	-0.8136
6	27255	0	0	1.248804	0.047208	0.423388	-0.13952	-0.59222	-0.98065	-0.04242	-0.12304	-0.08896	-0.03444	1.704529	1.074219	0.09154	0.451779
7	31944	1170	0	-2.29093	-2.57857	-0.3069	0.61881	-3.81479	2.482192	3.293415	0.441752	-0.12338	-1.71159	-1.56911	-0.9281	-0.19017	0.233115
8	33184	15.56	0	0.951961	0.416024	0.788296	2.341854	0.471227	1.333706	-0.15288	0.445539	-0.85856	0.457759	0.857263	0.718679	0.449485	0.323364
9	34118	76	0	1.453447	-0.97469	-0.13496	-1.41855	-1.05775	-0.94885	-0.46114	-0.35084	-2.14915	1.4041	-0.75852	-0.79507	0.834501	-0.16929
10	34897	8	0	1.331935	-0.39731	-0.11311	-0.69717	-0.57256	-0.70396	-0.17446	-0.12372	-1.42988	0.860998	1.581395	0.53885	-0.14306	0.624942
11	35181	0.89	0	1.214692	0.245851	0.309292	0.584285	-0.38636	-0.76711	-0.0743	-0.00872	-0.10002	-0.08878	1.48723	0.477046	-0.69641	0.075485
12	35484	31	0	1.39959	-0.5907	0.168619	-1.02995	-0.53981	0.040444	-0.71257	0.002299	-0.97175	0.756801	0.543827	0.112453	1.075384	-0.24577
13	36721	12.31	0	1.142349	0.090803	0.198756	1.031747	-0.58085	-0.62864	-0.28634	0.127945	0.560734	-0.55871	1.222848	0.092665	-1.9157	-1.03296
14	38238	380	0	-1.57263	0.612082	0.129682	-1.29043	-1.68131	0.188992	1.968965	0.081226	-0.44264	-0.75769	0.764797	0.391447	-0.17297	0.466562
15	38671	150.92	0	0.935768	-0.86388	0.965979	0.373065	-1.40666	-0.35306	-0.51479	0.060357	1.303544	-0.49607	-0.92077	0.178562	-0.7585	-0.37228
16	39394	281.94	0	-2.15194	-2.37882	1.904957	-2.28725	-0.33062	-0.97111	-0.06753	0.251995	2.391839	-2.6812	-0.59542	0.892896	-0.72211	-0.47807
17	41505	364.19	1	-16.5265	8.584972	-18.6499	9.505594	-13.7938	-2.8324	-16.7017	7.517344	-8.50706	-14.1102	5.299236	-10.834	1.67112	-9.37386
18	42145	17.33	0	-0.63611	1.236421	1.189131	0.676485	0.111772	-0.10726	0.642491	0.17567	-0.56416	0.416173	1.539318	0.493868	-0.55585	0.562179
19	42272	37.73	0	0.925523	-0.68796	1.015057	-0.05942	-0.33952	1.98513	-1.03381	0.776725	0.885371	-0.49563	1.907331	1.779557	0.082448	-0.28383
20	43985	101.19	0	-1.68217	0.341439	1.287377	-1.00005	-0.92961	0.037511	0.321687	0.254967	1.778829	0.140253	-0.97607	-0.20651	-1.33711	-0.73801
21	43994	53.5	0	-0.53679	0.545234	-0.15406	-1.05205	2.405064	3.556153	0.307192	0.931851	-0.34351	-0.37108	-0.48988	-0.16073	-0.19649	0.130012
22	44261	520.12	0	0.339812	-2.74375	-0.13407	-1.38573	-1.45141	1.015887	-0.52438	0.22406	0.899746	-0.56501	-0.08767	0.979427	0.076883	-0.21788
23	45102	26.88	0	-0.94068	0.903703	0.978734	-0.43772	0.876943	1.047219	0.859711	-0.24089	0.607975	0.867774	0.181979	0.37654	0.071096	-0.73203
24	49848	21.95	0	1.104574	-0.54013	1.127897	0.019039	-1.08371	0.308094	-0.91705	0.377067	0.969931	-0.2425	1.112838	0.731945	-0.97192	-0.03438
25	52840	69	0	-1.92939	-0.69317	1.197647	-2.36525	-1.3834	2.47E-05	-1.37881	1.077452	-2.68164	0.443854	0.526857	-0.51592	0.043383	0.296375
26	52913	0.89	0	0.676855	1.587029	-1.50815	1.443815	1.31679	-1.16034	1.149049	-0.41939	-0.09057	-0.32306	2.967921	-0.15503	-0.71777	-3.70928
27	54263	0.76	0	-0.25511	1.062676	0.521598	0.599852	0.660175	-1.16714	0.74838	-0.13967	-1.06528	-0.6023	1.916162	0.651058	0.336313	-0.92896
28	55575	22.7	0	-3.41942	-1.13008	1.607086	2.229979	2.644089	-0.79354	1.088242	-1.00841	-0.93332	1.921982	1.954445	0.38884	0.301474	-0.42787
29	56803	1	0	1.173609	0.511603	-0.52045	1.122626	0.358867	-0.29596	0.025005	0.144442	-0.11018	-0.51336	1.500906	-0.22873	-1.57033	-0.83968
30	58084	34.99	0	1.230866	0.095267	0.257008	0.535814	-0.31462	-0.70776	0.061866	-0.15429	0.150024	-0.06594	-0.74508	-0.05631	-0.0855	0.3152

출처: **Kaggle Credit Card Fraud Detection** (mlg-ulb)

파일: **dataset/creditcard.csv**
총 284,807건 거래 / 31개 변수

- 기간: 약 이틀(~172,792초) 분량의 실제 유럽 신용카드 거래 데이터

타깃: **Class** (0 = 정상, 1 = 사기) · 사기 건수 492건 (약 0.17%)

학습용 정식 입력은 CSV, 소표본 Excel은 **scripts/make_creditcard_sample_excel.py**

Kaggle: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

1-2 변수 구조와 역할

V1 ~ V28

- PCA 등으로 변환·익명화된 특성
- 원 변수명·의미 비공개
- 도메인 해석 불가
  → 패턴 탐지 중심

Time / Amount

- Time: 첫 거래 이후 경과 초 (단조 증가)
- Amount: 거래 금액 (꼬리 두꺼운 분포)
- 두 변수 모두 추가 전처리 필요

Class (타깃)

- 0 = 정상 (99.83%)
- 1 = 사기 (0.17%)
- 극심한 클래스 불균형

V1~V28 익명화로 도메인 지식 활용이 제한되므로 패턴 기반 모델링에 집중.

1-3 클래스 분포 확인 (불균형)

Imbalance

핵심 관찰

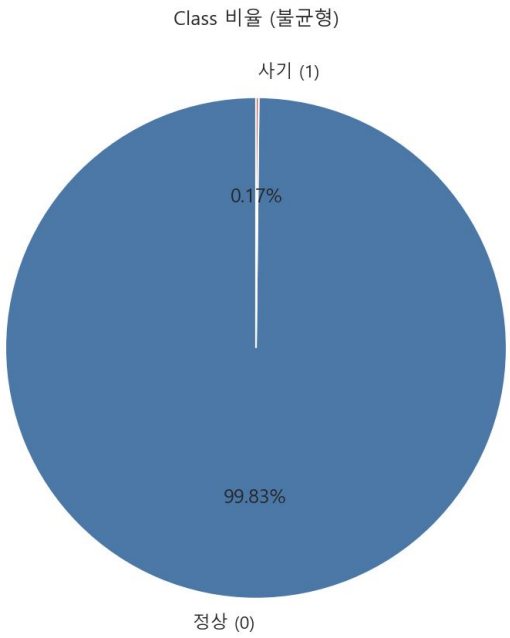
- 정상 284,315건 vs 사기 492건 (약 579:1)

"전부 정상 예측" 시 Accuracy 99.83% → **지표 무의미**

Recall · PR-AUC 중심 평가 필요

- SMOTETomek으로 Train 불균형 보정 예정

Visualization



불균형이 전처리 전략(SMOTETomek)과 평가 지표 선택의 출발점.

1-4 기초 통계 요약

Time·Amount (Class 별)

Class	변수	개수	평균	표준편차	최소	25%	50%	75%	최대
0	Time	284315	94838.2023	47484.0158	0	54230	84711	139333	172792
0	Amount	284315	88.291	250.1051	0	5.65	22	77.05	25691.16
1	Time	492	80746.8069	80746.8069	406	41241.5	75568.5	128483	170348
1	Amount	492	122.2113	122.2113	0	1	9.25	105.89	2125.87

Amount: 최솟값 0, 최댓값 수만 단위 → **꼬리 두꺼운 분포**

Time: 0~172,792초 단조 증가 → **주기 인코딩 필요**

기초 통계에서 Amount·Time 전처리 필요성을 확인.

제2장

탐색적 데이터 분석 (EDA)

불균형 · 상관 구조 · Time 분포 · Amount 분포

2-0 EDA 개요와 분석 항목

EDA Overview

- 목적: 전처리 선택(sin/cos · RobustScaler)과 모델 방향(트리 계열)의 근거 마련
- 분석 항목 4가지:
- 1) 클래스 불균형 → SMOTETomek 적용 근거
- 2) 상관 구조 → 트리 계열 모델 선택 근거 (PCA 직교화)
- 3) Time 분포 → 86,400초 주기 인코딩 근거
- 4) Amount 분포 → RobustScaler 선택 근거

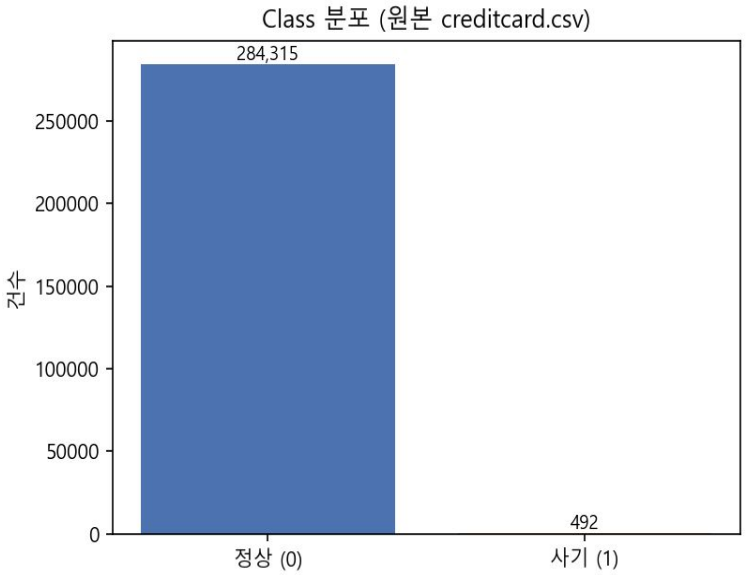
산출 스크립트: fds_preprocessing_walkthrough.ipynb · fds_report_figures.ipynb · report/generate_report_extras.py

EDA → 전처리 → 모델 선택의 인과 흐름을 4개 항목으로 구조화.

2-1 클래스 불균형 분석

Imbalance

📊 결과



📌 목적

Accuracy 단독 지표의 한계를 실증하고
SMOTETomek 적용의 필요성을 데이터로 확인

🔧 방법

Class별 건수 집계 · 비율 계산
막대/파이 차트로 시각화

💡 해석 → 연결

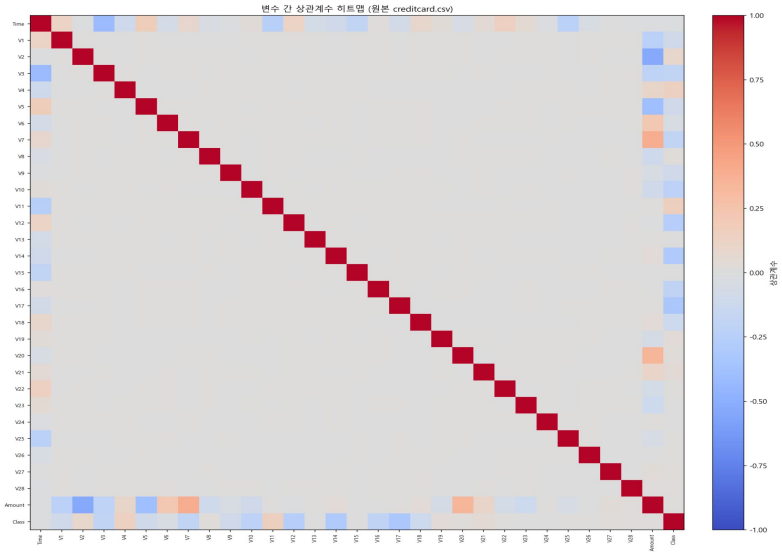
"전부 정상" 예측으로도 Acc 99.83%
→ Recall·F2·PR-AUC 지표 채택
→ SMOTETomek(Train에만) 적용

이 분석이 지표 선택(제6장)과 리샘플링(제4장)의 직접 동기.

2-2 상관 구조와 타깃 연관

Correlation

📊 결과



📌 목적

V1~V28 간 다중공선성 여부와
Class와의 연관 신호를 탐색

🔧 방법

Pearson 상관 히트맵
(V1~V28 · Amount · Time · Class)

💡 해석 → 연결

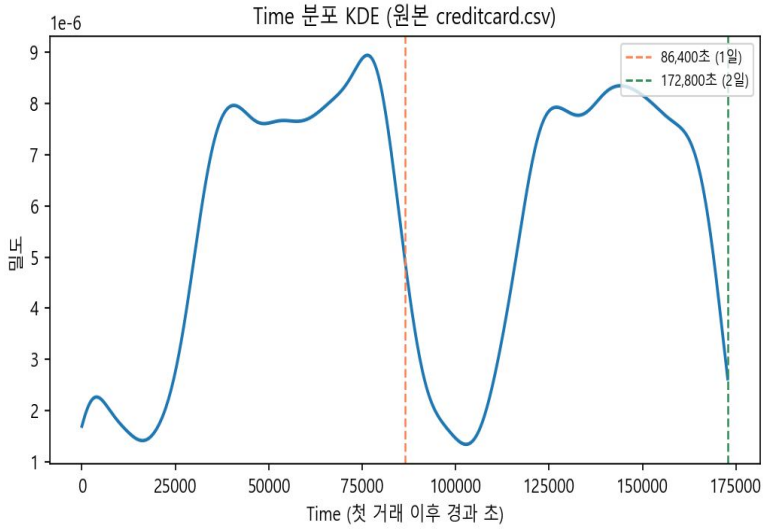
다중공선성 부재 → 트리 계열 적합
원 변수 의미 해석 불가
→ 상대적 패턴 탐지 목적으로만 활용

이 분석이 지표 선택(제6장)과 리샘플링(제4장)의 직접 동기.

2-3 Time 분포 → 주기 인코딩 동기

Time

📊 결과



📌 목적

Time 단조 증가값을 그대로 쓰면
발생할 과적합 위험을 확인

🔧 방법

Time KDE · 시간대별 밀도 시각화
86,400초(1일) 기준 주기성 탐색

💡 해석 → 연결

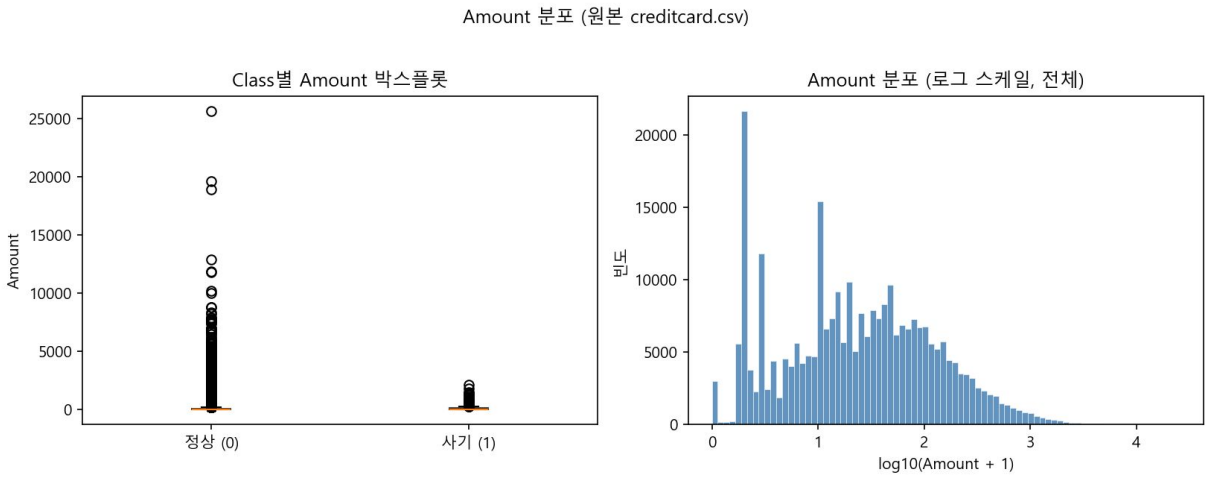
단조 증가 → 트리 구간 과적합 위험
→ sin/cos(86,400s) 주기 인코딩
→ 원 Time 컬럼 삭제

이 분석이 지표 선택(제6장)과 리샘플링(제4장)의 직접 동기.

2-4 Amount 분포 → RobustScaler 동기

Amount

📊 결과



📌 목적

Amount 분포 특성(꼬리 두꺼움)을 확인하고 스케일러 선택 근거 마련

🔧 방법

히스토그램 + 박스플롯
(Class별 분리 포함)

💡 해석 → 연결

평균·표준편차 기반 스케일러는 이상치에 민감
→ RobustScaler 선택
(중앙값·IQR 기반으로 이상치 강건)

이 분석이 지표 선택(제6장)과 리샘플링(제4장)의 직접 동기.

제3장

전처리 파이프라인 (SSOT)

fds_pipeline.py · 단계별 입출력 · 재현성 보장

3-0 SSOT: fds_pipeline.py 개요

Single Source of Truth

모든 학습·실험 스크립트가 **fds_pipeline.py**의 동일 전처리 함수를 호출

- 재현성 보장: 동일 환경에서 동일 결과 (random_state 고정)

노트북 대응: **fds_preprocessing_walkthrough.ipynb**이 동일 순서(학습 제외)를 따름

핵심 함수: **preprocess_creditcard_dataframe()** / **train_xgb_pipeline()**

① CSV 로드 — ② Time → sin / cos — ③ Amount RobustScaler — ④ X/y 분리 — ⑤ 학습 단계로

⚠ 스케일러 fit: 코드상 분할 전 전체 데이터 적용 — 누출 논의: docs/leakage_and_scaling.md

3-1 Time → sin/cos 주기 인코딩

Step ①

Time Cyclic Encoding

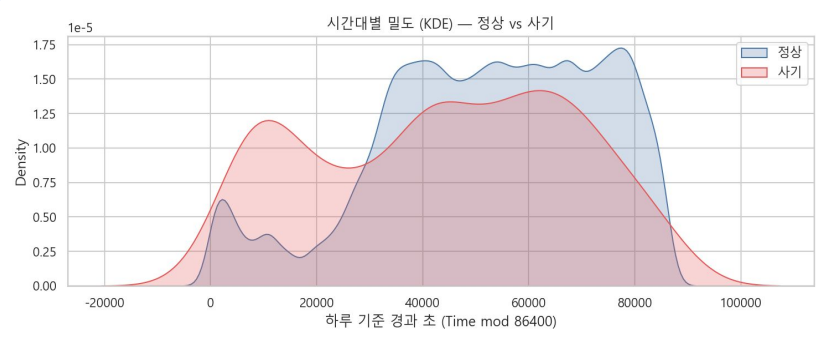
입력: **Time** 컬럼 (0 ~ 172,792 초, 단조 증가)

변환: **Time_sin = sin(2π × Time / 86400)**

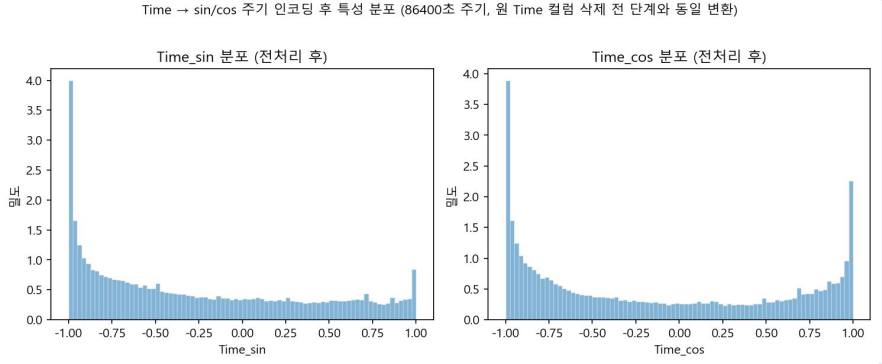
Time_cos = cos(2π × Time / 86400)

출력: **Time_sin, Time_cos** 생성 → 원 Time 컬럼 삭제

변환 전



변환 후



단조 증가 → 주기 원형 표현으로 트리 과적합 위험 완화.

3-2 Amount → RobustScaler 적용

Step ②

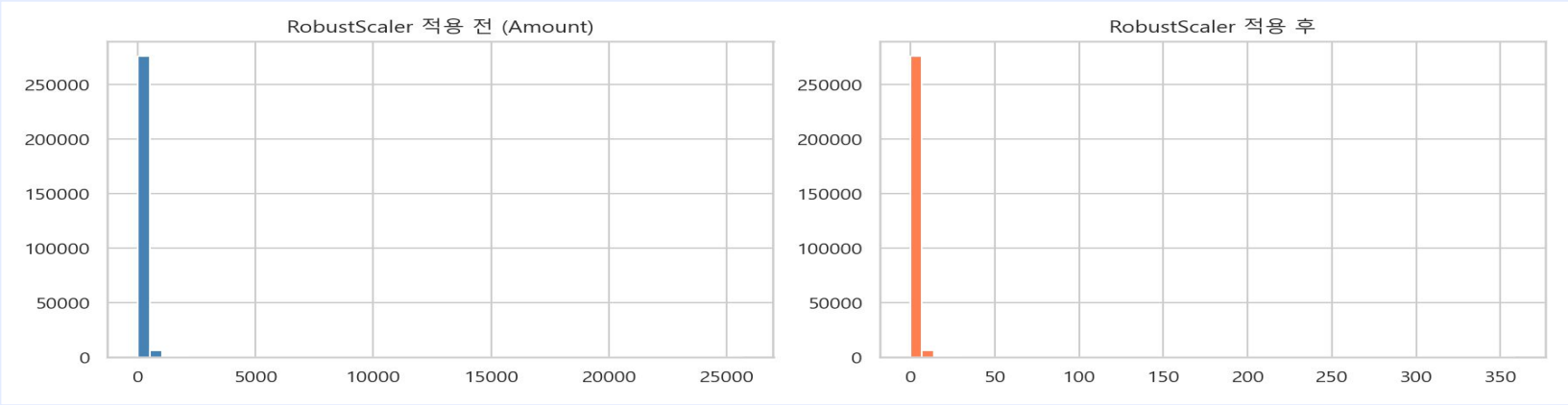
Amount Scaling

입력: **Amount** 컬럼 (0 ~ 25,691.16, 꼬리 두꺼운 분포)

스케일러: **RobustScaler** (중앙값·IQR 기반 → 이상치에 강건)

fit 시점: 코드상 **분할 전 전체 데이터**에 fit → 누출 트레이드오프 존재 (docs/leakage_and_scaling.md)

- 출력: 스케일된 Amount 컬럼 (이상치 영향 최소화)



RobustScaler: StandardScaler 대비 이상치 영향 최소화 효과 확인.

3-3 특성·타깃 분리와 파이프라인 요약

Step ③

Feature / Target Split · Pipeline Summary

X 특성: V1~V28 + Amount(스케일됨) + Time_sin + Time_cos → **30개**

- y 타깃: Class (0/1) — 분리 후 이후 분할·리샘플 단계로 전달

노트북: **fds_preprocessing_walkthrough.ipynb** — 동일 순서 수행 (학습 제외)

단계	입력	출력	비고
1. CSV 로드	creditcard.csv	DataFrame(284807×31)	
2. Time 인코딩	Time 컬럼	Time_sin, Time_cos (Time 삭제)	sin/cos 86400s
3. Amount 스케일	Amount 컬럼	Amount_scaled	RobustScaler
4. X/y 분리	전체 DataFrame	X(30개), y(Class)	

파이프라인 요약표. 수치는 코드 실행 결과로 확인.

제4장

데이터 분할과 리샘플링

층화 분할 · SMOTETomek · 누출 논의

4-1 층화 분할 설계

Stratified Train/Test Split

함수: **train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)**

- Train 80% ≈ 227,845건 · Test 20% ≈ 56,962건
- stratify=y: Class 비율(0.17%)을 Train/Test 양쪽에서 유지

메인 파이프라인은 **Train / Test 2분할만** 사용

전체 데이터	Train	Test
284,807건	227,845건 (80%)	56,962건 (20%)

📌 fit | val | test 3분할은 model_comparison_round2.py (2차 실험)에만 해당
메인 파이프라인·1차 벤치마크는 Train / Test 구조를 유지합니다.

4-2 SMOTETomek 리샘플링

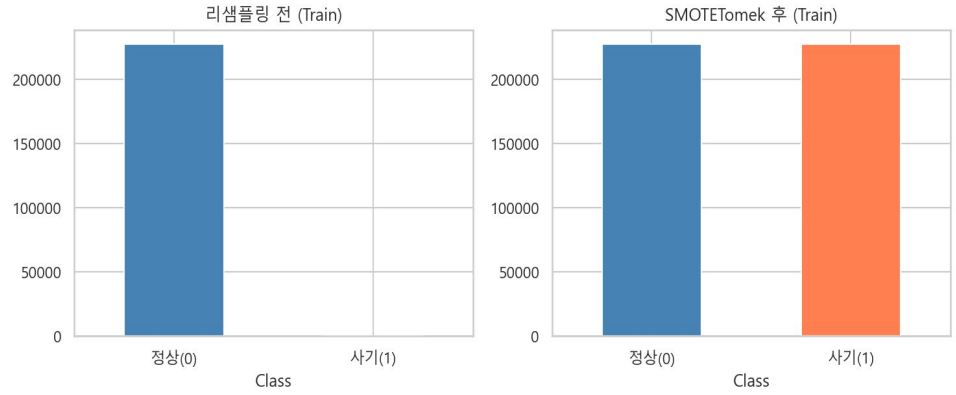
왜 Train에만?

- SMOTE: 소수 클래스 합성 오버샘플링
- Tomek: 경계 노이즈 링크 제거
- Train에만 적용: Test 누출 방지

Test: **원 불균형 분포 유지**
→ 실운영 환경 모사

함수:
SMOTETomek(random_state=42)

리샘플링 효과



구분	Class 0	Class 1
리샘플 전	227,451	394
SMOTETomek 후	227,451	227,451

SMOTETomek은 Train에만. Test는 원 불균형 — 실운영 환경 모사.

4-3 스케일러 fit 시점과 누출 논의

Data Leakage Discussion · docs/leakage_and_scaling.md

현재 구현: **분할 전 전체 데이터에 fit** → Test 분포 정보가 스케일러에 노출될 수 있음

- 대안: Train에만 fit → Test에 transform만 적용 (엄밀 기준)

트레이드오프: creditcard.csv는 공개 데이터라 실질 누출 위험은 낮으나, **원칙상 논의 포인트**

상세 논의: **docs/leakage_and_scaling.md** 참고

비교 실험:
- **scripts/compare_preprocessing_strategies.py** - RobustScaler full vs train-only

- Train-only fit과의 수치 비교는 scripts/compare_preprocessing_strategies.py 실험 1(표)에 포함되어 있다.

⚠ 이 한계는 제5장 전처리 민감도 실험(compare_preprocessing_strategies.py)에서 보완적으로 검토.

제5장

모델링과 실험

베이스라인 · 1차 벤치마크 · 2차 실험 · 전처리 민감도

5-0 실험 구조 한눈에 보기

메인·1차는 Train/Test 2분할 · **fit|val|test 3분할은 2차 실험에만**

메인 파이프라인
fds_pipeline.py
(Baseline)

- Train/Test 2분할
- SMOTETomek: Train만
- XGB 베이스라인 고정
- fds_*.pkl 아티팩트 저장

1차 벤치마크
model_comparison.py

- 동일 전처리·동일 분할
- 최대 5종 모델 비교
- 임계값 기본 0.5
- ROC/PR/혼동행렬 시각화

2차 실험
model_comparison_round2.py

- fit|val|test 3분할
- SMOTETomek: fit만
- val→F2 최대 임계값
- test 최종 평가

⚠ **1차와 2차는 학습 표본 범위가 달라 수치 직접 비교 금지 (코드 주석과 동일 메시지)**

5-1 재현 기준선: XGBoost Baseline

Baseline Model · train_xgb_pipeline()

train_xgb_pipeline(): 전처리 → 분할 → SMOTETomek → 학습 일괄 수행

하이퍼파라미터: **XGB_BASELINE_PARAMS** 고정 (n_estimators=100, learning_rate=0.1 등)

아티팩트 저장: **scripts/train_save_artifacts.py**

→ **fds_model.pkl · fds_scaler.pkl · fds_X_test.pkl · fds_y_test.pkl**

Recall	F1	F2	PR-AUC	ROC-AUC
0.857143	0.597865	0.730435	0.851009	0.974444

재현 가능한 고정 기준선 확보. 수치지표는 model_comparison.py 산출

5-2 1차 벤치마크 설계

model_comparison.py · run_benchmark()

- 조건: 동일 전처리 · 동일 Train/Test 분할 · SMOTETomek(Train) · 임계값 0.5
- 비교 모델 (최대 5종):
- 1) 로지스틱 회귀 (기본)
- 2) 로지스틱 회귀 (RandomizedSearchCV 튜닝)
- 3) 랜덤포레스트 (고정 하이퍼)
- 4) XGBoost (기본 = 베이스라인)
- 5) XGBoost (RandomizedSearchCV 튜닝)

시각화 일괄 출력: scripts/model_comparison_visualize.py
→ report_figures/ (ROC · PR · 막대 · 혼동행렬 · 보정곡선 · 임계값-성능 곡선 + CSV)

5-3 1차 벤치마크 결과

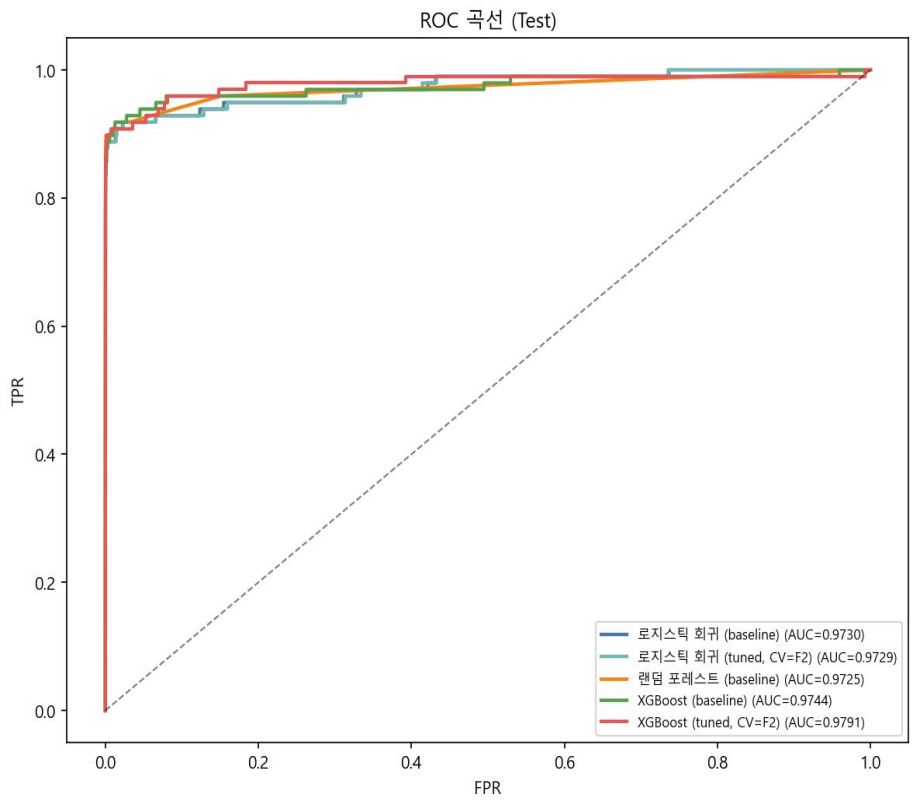
임계값 0.5 기준 · 수치는 코드 실행 결과로 채움

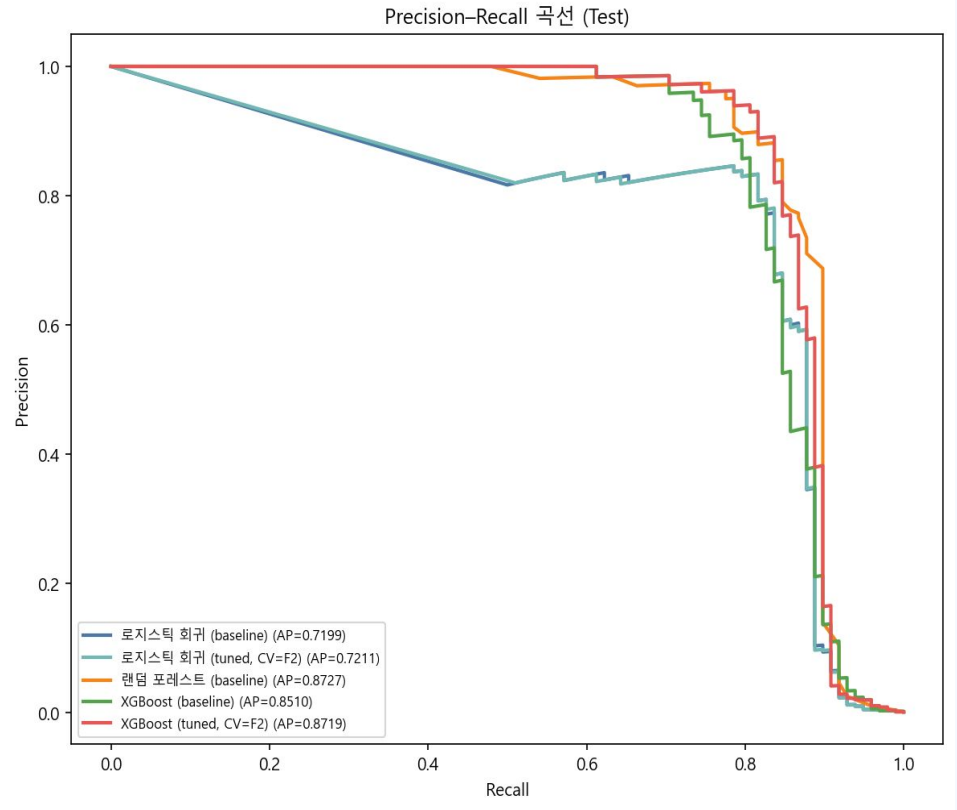
Benchmark Table (1차)

모델	설정	Recall	F1	F2	PR-AUC	ROC_AUC
로지스틱	기본	0.918367	0.108173	0.229826	0.719884	0.973026
로지스틱	튜닝	0.918367	0.108043	0.229592	0.721109	0.972904
랜덤포레스트	고정	0.816327	0.851064	0.829876	0.872667	0.972507
XGBoost	기본	0.857143	0.597865	0.730435	0.851009	0.974444
XGBoost	튜닝	0.816327	0.812183	0.814664	0.865765	0.980858

5-4 ROC·PR 곡선(범례 AUC/AP)

Visualize

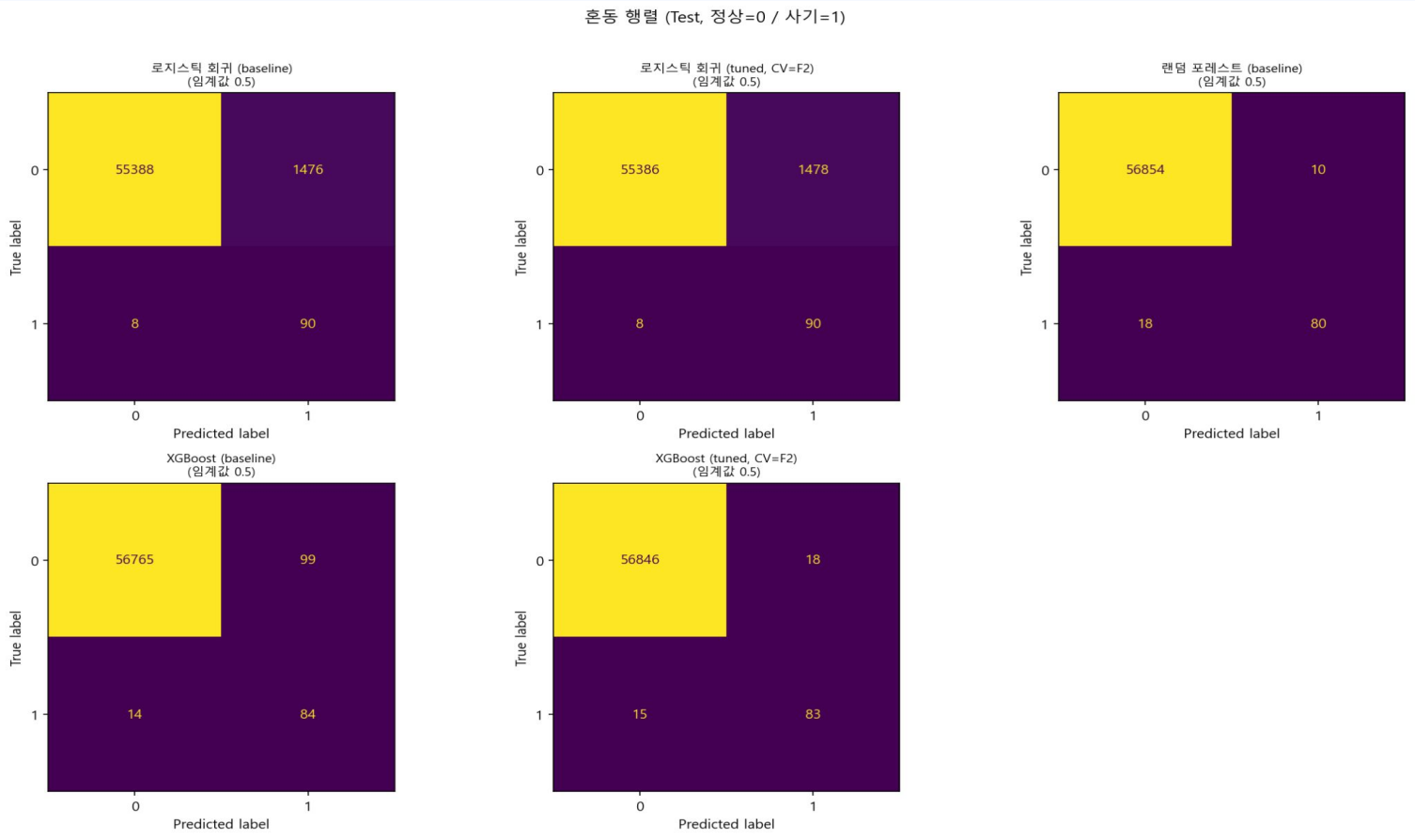




산출: scripts/model_comparison_visualize.py → report_figures/

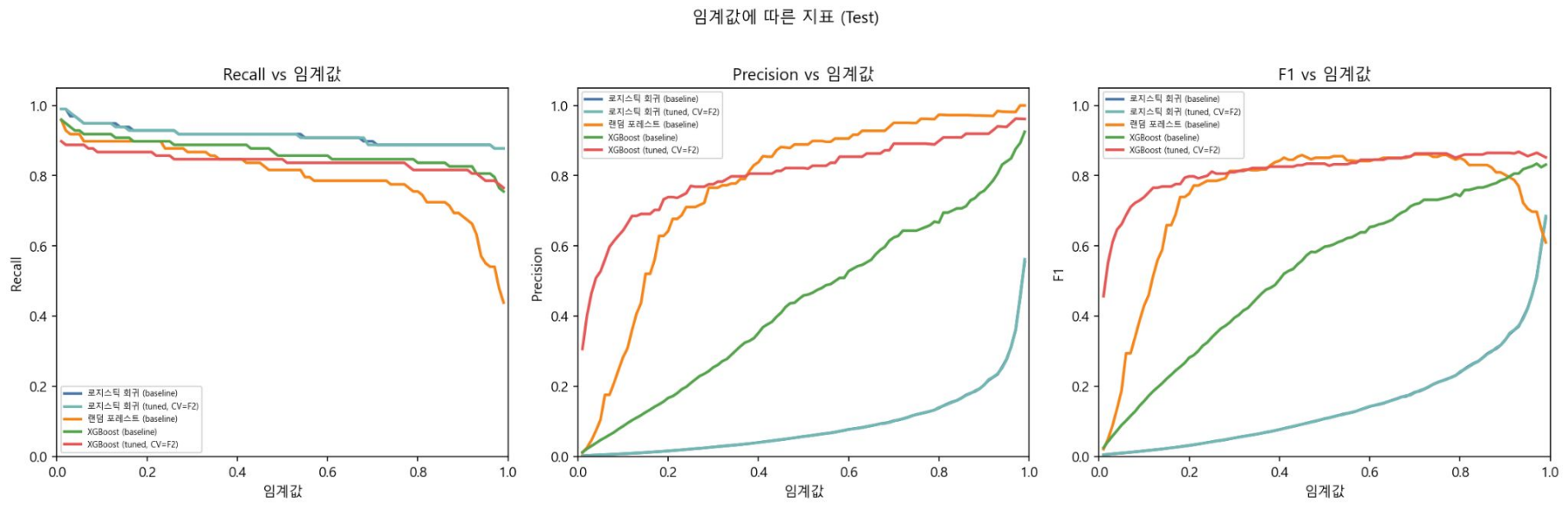
5-4 혼동행렬 5종 (thr=0.5)

Visualize



5-4 임계값별 Recall / Precision / F1

Visualize



5-5 플로우차트 다이어그램

Diagram

Key points

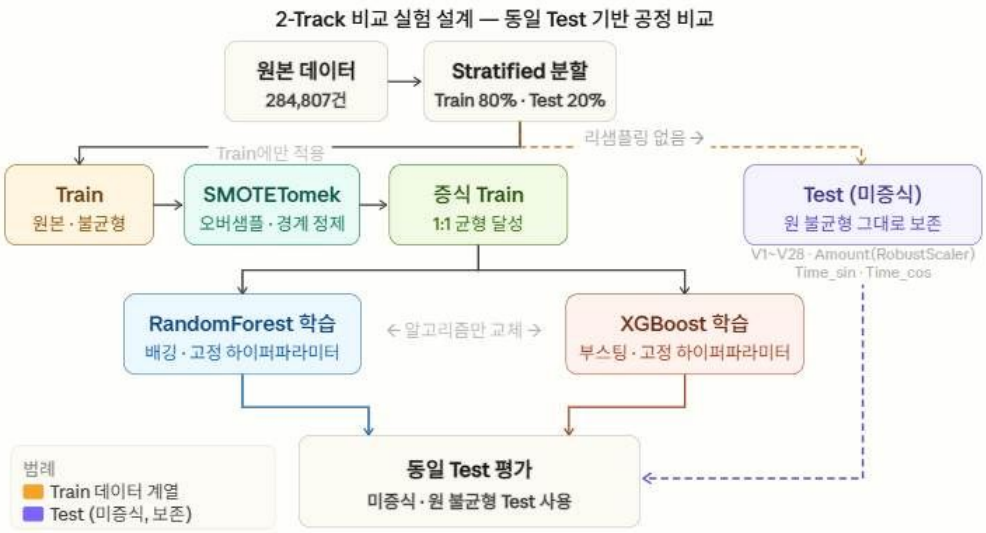
- 2-Track = RF(배깅) vs XGB(부스팅) 개념 비교

수치 표(슬라이드 5-3)와 **목적이 다름** — 직접 수치 비교 불가

- 3-fold CV Recall 학습곡선: 데이터 크기 변화에 따른 안정성 확인

출처: **report/generate_report_model_diagrams.py**

Visualization



도식은 개념 설명용 — 5종 벤치마크 수치 표와 섞어 해석하지 말 것.

5-6 2차 실험 설계

model_comparison_round2.py · 층화 분할(fit·val·test) · Threshold Selection

이 파일에서만 Train → **fit / val / test 3분할** 적용

SMOTETomek: **fit에만** (val·test는 원 불균형 분포 유지)

val에서 **F2 최대** 임계값 탐색 → test 최종 평가 1회

- 대상 모델: RF 베이스 · XGB 베이스 · XGB 튜닝 (3종)

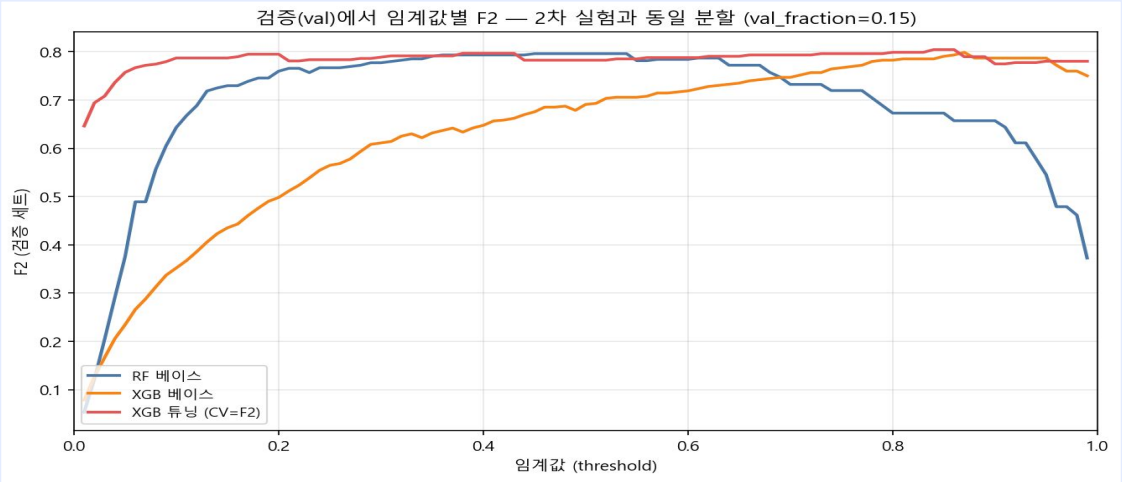
⚠ 1차 결과와 수치 직접 비교 금지 — 학습 표본 범위 상이 (코드 주석과 동일 메시지)

5-7 2차 실험 결과

val 최적 임계값 기준 · test 최종 평가 · 수치는 실행 결과로 채움

Benchmark Table (2차 · val 최적 임계값 적용)

모델	설정	val 임계값	test Recall	test F2	test PR-AUC
RF	베이스	0.45	0.826531	0.833333	0.866707
XGBoost	베이스	0.87	0.816327	0.816327	0.819797
XGBoost	튜닝	0.84	0.826531	0.845511	0.868770



5-8 전처리 민감도 비교

compare_preprocessing_strategies.py

- 목적: 스케일러 fit 시점·리샘플링 전략 변경이 성능에 미치는 영향 측정

비교 ①: **RobustScaler full-fit** (현재) vs **train-only fit**

비교 ②: **SMOTETomek** vs **scale_pos_weight (XGB 내장)**

- 동일 평가 지표(Recall / F2 / PR-AUC) 기준으로 차이 측정

스케일 방식	recall	f1	f2	PR_AUC
full-fit	0.857143	0.597865	0.730435	0.851009
train-only	0.857143	0.600000	0.731707	0.824311

불균형 처리	recall	f1	f2	PR_AUC
SMOTETomek	0.857143	0.597865	0.730435	0.851009
scale_pos_weight	0.836735	0.811881	0.826613	0.864908

- 스케일러 fit 시점에 따른 차이는 아래 표와 함께 docs/leakage_and_scaling.md에서 해석한다.
- 동일 stratified split (random_state=42, test_size=0.2), Test 지표, 임계값 0.5, 모델은 동일 XGB 베이스라인

제6장

평가 지표와 임계값

Recall · F2 · PR-AUC · threshold 역할

6-1 평가 지표 정리

Recall (재현율)	실제 사기 중 탐지 비율 → 미탐(FN) 최소화 핵심
F2 Score	Recall 2배 가중 - 오탐보다 미탐 피해가 큰 금융 도메인에 적합
PR-AUC	y_prob 전체 순위 기반 · 불균형에 민감
F1 / Accuracy	벤치마크 출력 포함 가능. 단, Accuracy 단독 지양 임계값(Threshold) 바꾸면 Recall · Precision · F1 · F2는 달라짐
임계값 (threshold)	PR-AUC는 y_prob의 순위만 쓰므로 임계값과 무관하며, 같은 예측 확률 벡터에 대해서는 값이 변하지 않는다

앱에서 임계값 슬라이더 변경 시: Recall · F2 재계산 / PR-AUC 고정 표시

6-2 임계값(Threshold) 역할과 조정

Threshold

임계값 조정 원리

- **판정**
  - y_prob ≥ threshold → 사기(1). app.py는 테스트 전체·단건 모두 같은 규칙

- **임계값 조정**
  - 임계값 ↓: 사기로 더 많이 잡음 → Recall↑, Precision↓ (미탐↓·오탐↑ 경향)
  - 임계값 ↑: Precision↑, Recall↓

- **PR-AUC**
  - average_precision_score(y_test, y_prob) - 확률만 쓰므로 임계값과 무관·고정 (app 사이드바 표시와 동일)

- **2차 실험**
  - model_comparison_round2.py → val에서 0.01~0.99 스캔해 F2 최대인 threshold 선택 → 그 값으로 test 한 번 평가. SMOTETomek은 fit만. 1차 수치와 직접 비교 금지
  - 앱: 슬라이더(0.01~0.99)로 threshold를 바꾸면 Recall·F2는 재계산되고, PR-AUC는 그대로 표시

비즈니스 비용(미탐 vs 오탐)에 따라 임계값을 맞춤 조정하는 것이 운영 포인트.

제7장

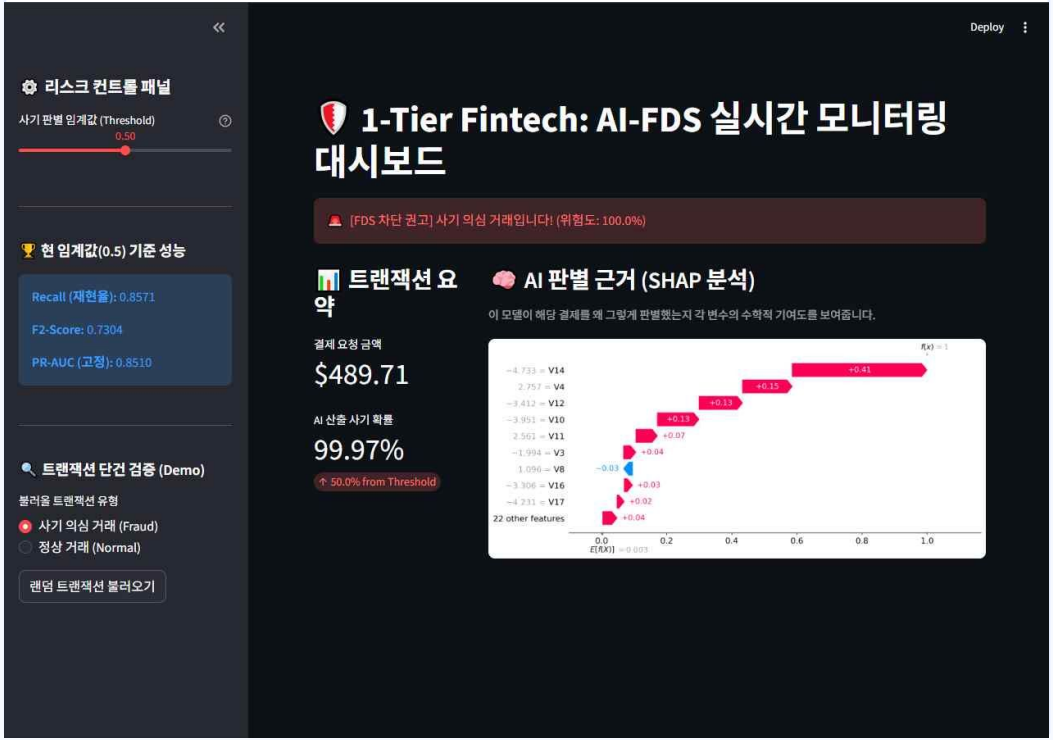
Streamlit 앱과 SHAP 설명

아티팩트 로드 · 시연 화면 · Explainer 구현

7-1 Streamlit 앱 구조와 아티팩트 로드

App Structure

Dashboard



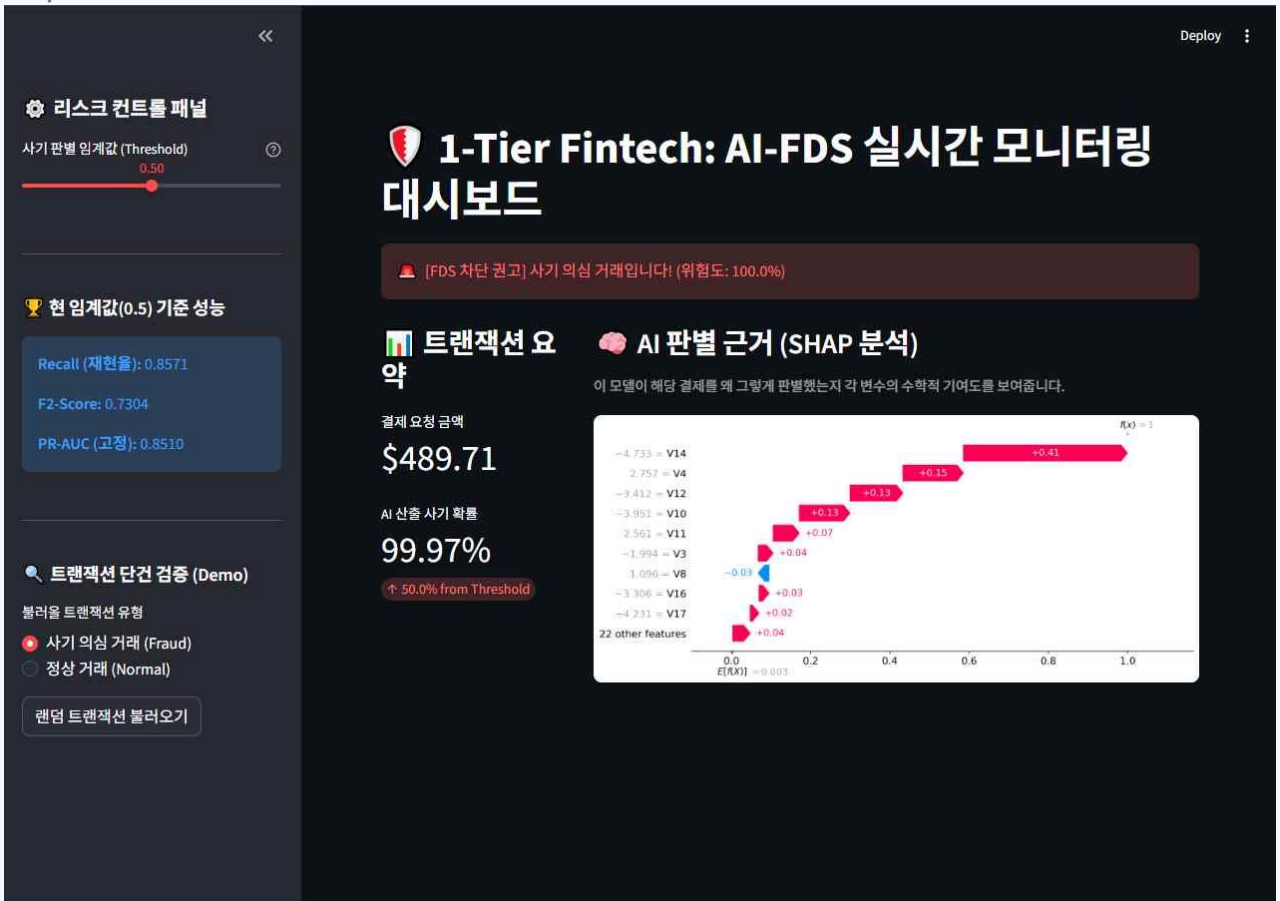
Load & Run

- pkl 존재 시: joblib으로 로드만
- 없으면: 시작 시 전체 학습 자동
- **fds_model.pkl**
- **fds_scaler.pkl**
- **fds_X_test.pkl · fds_y_test.pkl**

실행: **streamlit run app.py**

7-2 시연 화면

임계값 조정 · Recall/F2 재계산 · PR-AUC 고정 · 단건 예측 · SHAP 흐름



7-3 SHAP 구현 방식과 주의사항

Explainability — App Implementation

앱 구현: **predict_proba 래핑** + **shap.Explainer(배경 샘플)**

- 배경 샘플: Test 세트 일부 — base value(기준값) 추정에 사용

TreeExplainer 직렬화 이슈 회피를 위한 함수형 Explainer 선택

⚠ 보고서·슬라이드에 TreeExplainer만 명시 시 앱 구현과 **불일치**

V1~V28 익명화 → 절대 기여값보다 **상대적 기여 순위** 중심 해석

✅ **올바른 명시: "앱은 predict_proba 래핑 + shap.Explainer(배경 샘플)"**

설명 가능성은 개별 예측 이해 보조 수단 — 도메인 해석에는 한계.

7-4 SHAP 워터폴: 단건 설명에만 초점

SHAP

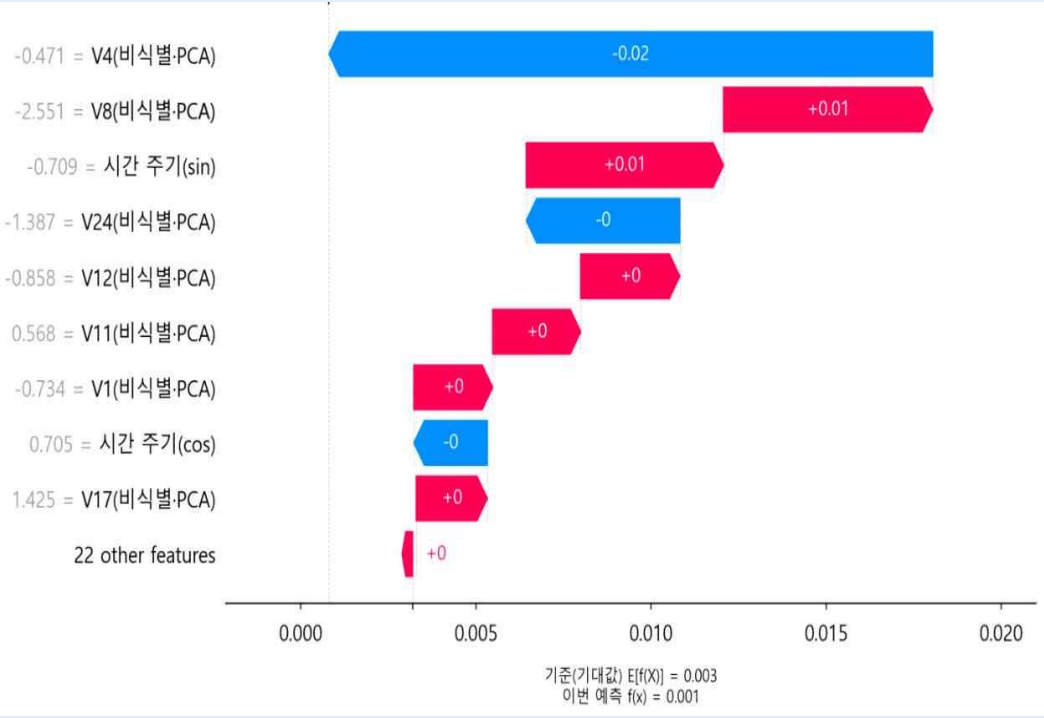
Key points

- 각 특성이 예측 확률을 얼마나 높이거나 낮추는지 표시
- 양의 기여(사기 확률 ↑) / 음의 기여(사기 확률 ↓)

V1~V28 익명화
→ **상대적 기여 순위** 중심

- 단건 데모: 정상/사기 선택 → 무작위 1건 → 워터폴 표시

Visualization



"왜 이 거래를 사기로 봤는가"를 모델이 스스로 설명하는 장면.

제8장

한계와 향후 계획

익명화 · OOT · 프로토타입 범위

8-1 현재 한계

Limitations

1 **익명화 특성**

V1~V28은 PCA 기반 → 원 변수 의미 해석 불가. 도메인 지식 활용 제한

2 **OOT 검증 부재**

- 시간 외 검증(Out-of-Time)은 이 데이터 구조(이틀치)로 엄밀히 설계 어려움
- Time이 단조인 반면 데이터가 약 이틀 분량이라 엄밀한 시간 외 검증 설계가 제한되며, 향후 방향은 8-2에 명시

3 **스케일러 누출 논의**

공개 벤치마크 재현·파이프라인 단순화와의 트레이드오프로 선택했으며, 엄밀 기준은 Train-only fit과의 비교 실험(5-8, compare_preprocessing_strategies.py)으로 보완적으로 검토

4 **프로토타입 범위**

실운영 연동(MSA·Kafka·실시간 스코어링)은 설계 범위 밖. 추가 단계 필요

8-2 향후 계획

Next Steps

- **우선 순위**

스케일 정책(Train-only fit) 재현 실험·수치 정리 → 시간 기반 분할(롤링 등) OOT 대리 검증 프로토타입

- 스케일러 fit 시점 재검토: Train-only fit 적용 후 성능 비교 실험 (compare_preprocessing_strategies.py 활용)
- OOT 대리 검증: Time 기반 롤링 윈도우 분할로 시간 연속성 반영 실험
- 추가 모델 탐색: LightGBM · CatBoost · 앙상블 시도
- 임계값 정교화: 금융 비용 함수 정의 후 최적 임계값 산출 (미탐 비용 > 오탐 비용 가중)
- 실운영 연동 (장기): FastAPI 래핑 → MSA 연동 → Kafka 실시간 스트리밍 파이프라인

부 록

실행 명령어 · 파일 맵 · 보고 그림 대응표 · 노트북 · 참고 링크

A-1 실행 명령어

Command Reference

python scripts/train_save_artifacts.py	아티팩트 저장 (fds_*.pkl)
streamlit run app.py	시연 앱 실행
python scripts/model_comparison.py	1차 벤치마크 (최대 5종)
python scripts/model_comparison_visualize.py	1차 시각화 일괄 출력 → report_figures/
python scripts/model_comparison_round2.py	2차 실험 (3분할 · F2 임계값)
python scripts/compare_preprocessing_strategies.py	전처리 민감도 비교
python report/generate_report_model_diagrams.py	2-Track · 학습곡선 · 배깅/부스팅 그림
python report/generate_report_extras.py	추가 보고 그림
python scripts/make_creditcard_sample_excel.py	소표본 Excel 생성 (학습 아님)
jupyter notebook fds_preprocessing_walkthrough.ipynb	전처리 순서 확인 노트북

A-2 주요 파일·폴더 맵

File Map

fds_pipeline.py	전처리·학습 SSOT — 모든 실험이 공유
app.py	Streamlit 시연 앱
artifacts/fds_model.pkl	학습된 XGBoost 모델
artifacts/fds_scaler.pkl	RobustScaler 객체
artifacts/fds_X_test.pkl · fds_y_test.pkl	Test 세트
docs/leakage_and_scaling.md	누출 트레이드오프 논의 문서
fds_preprocessing_walkthrough.ipynb	전처리 순서 노트북
fds_report_figures.ipynb	보고용 그림 노트북
report/generate_report_model_diagrams.py	2-Track·학습곡선·배깅/부스팅 다이어그램
report_figures/	시각화 산출 디렉토리 (CSV 포함)

A-3 보고 그림 대응표

Figure-to-Slide Mapping

산출 스크립트/노트북	그림·표 내용	대응 슬라이드
fds_preprocessing_walkthrough.ipynb	Time KDE · Amount 분포 · 리샘플 전후 건수	2-3, 2-4, 4-2
fds_report_figures.ipynb	상관 히트맵 · 클래스 분포	1-3, 2-1, 2-2
model_comparison_visualize.py	ROC · PR · 혼동행렬 · 임계값-성능 곡선	5-4, 6-2
model_comparison.py	1차 벤치마크 수치 표	5-3
model_comparison_round2.py	2차 실험 결과 표 · F2 임계값 곡선	5-7
generate_report_model_diagrams.py	2-Track 다이어그램 · CV 학습곡선	5-5
compare_preprocessing_strategies.py	전처리 전략별 성능 비교	5-8
app.py	SHAP 워터폴 스크린샷	7-4

A-4 노트북과 파이프라인 대응

Notebook ↔ Pipeline Correspondence

fds_preprocessing_walkthrough.ipynb: 전처리·분할·리샘플 순서 (학습 제외)

→ fds_pipeline.py의 전처리 함수와 **동일 순서** 수행

- → 슬라이드 3-0~3-3 (전처리 파이프라인 장)에 그림 자리와 대응

fds_report_figures.ipynb: 보고용 그림 각도 (EDA 슬라이드에 대응)

- → 슬라이드 1-3, 2-1~2-4의 이미지 자리에 산출물 삽입

두 노트북 모두 fds_pipeline.py의 함수를 import해 **동일 전처리 보장**

A-5 참고 링크와 레퍼런스

References

Kaggle 데이터셋 Credit Card Fraud Detection (mlg-ulb)	https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud
누출 논의 스케일러 fit 시점과 트레이드오프	docs/leakage_and_scaling.md (로컬)
imbalanced-learn SMOTETomek 문서	https://imbalanced-learn.org/stable/
SHAP shap.Explainer 함수형 API	https://shap.readthedocs.io/
Streamlit 시연 앱 프레임워크	https://streamlit.io/